Agentic AI System Design Report

Multi-Agent Workflow with In-Process Tool Server

Project 6 · Capstone · Sandip Dey

GitHub: https://github.com/sandipdey2/agentic-rag-system

Overview

This system answers questions about a 10-fact public academic knowledge base and performs simple arithmetic. An Orchestrator (an LLM when an API key is present, otherwise a rule-based fallback) makes a single routing decision to one of three specialized agents: RAG, Calculator, or Guardrail. Tools are invoked through an in-process class that follows the same register–list–call pattern as the Model Context Protocol (MCP). The system is bounded to the knowledge base and arithmetic; it refuses general-knowledge or instruction-override requests.

System Architecture and Design

The graph starts at the Orchestrator, which emits one routing decision. There is no Agent Gateway and no multi-step Plan-Act-Observe loop. The three specialized agents are:

- **RAG Agent** — retrieves a fact from the knowledge base via TF-IDF.
- **Calculator Agent** — evaluates a sanitized arithmetic expression.
- **Guardrail Agent** — returns a refusal for out-of-scope or injection queries.

A Reviewer node then appends a short confirmation. Session messages and a step trace are stored in the LangGraph state. The tool server is an ordinary Python class (`MCPToolServer`) that registers tools, lists them, and sanitizes responses; it is not a separate MCP process. An architecture diagram that matches this description is included as `architecture_diagram.pdf`.

Why a Multi-Agent Design

Separate agents are used because the two tools have different input contracts and failure modes. The RAG tool expects a natural-language query and returns a text fact; the Calculator expects a numeric expression and rejects non-arithmetic characters. Isolating each tool behind its own agent makes the Orchestrator's routing decision explicit and keeps the sanitization logic next to the tool that needs it. A single agent that called both tools would still need the same branching logic; the multi-agent graph makes that branching visible in the trace and easier to evaluate.

Decision Logic and Behavior

The Orchestrator is the only decision maker. On knowledge questions it selects the RAG tool; on arithmetic it selects the Calculator; on out-of-scope or injection attempts it routes to the Guardrail. Observed examples from the notebook:

- "What is a GAN?" returns the exact knowledge-base fact.
- "What is 12 * 8 + 5?" returns `Result: 101`.
- Weather and prompt-injection queries are refused.

The Reviewer only checks that a non-empty answer was produced; it does not perform a semantic entailment check against the retrieved context.

Observed Behavior and Evaluation

On five held-out queries the system achieved 4/5 (80%). Successful cases were the Variational Autoencoder fact, the RAG fact, the arithmetic query (Result: 69), and an out-of-scope refusal. The MNIST query was incorrectly refused by the Orchestrator, even though a matching fact exists in the knowledge base. This failure is reported rather than hidden.

RAGAS-style Metrics

Simplified local metrics inspired by the RAGAS framework (Es et al., 2023) were computed on the final answers:

- Faithfulness: 0.67
- Answer Relevance: 0.08

- Context Precision (approx): 0.67

Faithfulness is the fraction of answer tokens that also appear in the retrieved context. On the three answered knowledge queries the score is 1.0 by construction, because the answer is the retrieved fact. The 0.67 average is produced solely by the MNIST refusal (which contributes 0). It is therefore not independent evidence of grounding quality.

Safety, Reliability, and Transparency

The Orchestrator prompt limits the system to the knowledge base and arithmetic. The Calculator rejects any character that is not part of a simple arithmetic expression. Every tool call and the final answer appear in the printed trace. No API keys are stored in the repository; the key is supplied at runtime through the environment.

Ethical and Responsible Use Considerations

The MNIST failure illustrates a concrete accountability risk: the system produced a confident refusal for an in-scope question. A user who trusts the refusal would never see the correct fact that exists in the knowledge base. Because the Reviewer only checks answer presence, it marked the refusal as acceptable. This combination—an incorrect refusal plus a reviewer that does not verify grounding—can hide errors and reduce the user's ability to detect them.

Limitations and Future Improvements

Limitations include the small static knowledge base, occasional routing errors on short topic names (shown by the MNIST failure), and a reviewer that does not check semantic faithfulness. Realistic next steps are a stronger routing prompt that includes topic examples, embedding-based retrieval, and a reviewer that performs a simple token-overlap or entailment check against the tool output.

References

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

Wooldridge, M. (2009). *An introduction to multiagent systems* (2nd ed.). John Wiley & Sons.